# iOS Release Runbook: Setup & Team Updates

Nov 5, 2026 · @Robert Conde

## Who does what

The App Manager sets up each app once and owns anything touching Apple. Students build in the cloud from any OS, and one student per team who has a Mac uploads the builds with Transporter. The App Manager doesn't handle uploads.

| Who | Does | Needs |
|---|---|---|
| **App Manager** | Sets up each new app, manages testers, handles Apple-side changes | App Manager role with **Access to Certificates, Identifiers & Profiles** and **Create Apps** |
| **UF Admin** (e.g. the Account Holder) | Adds each team's uploader to App Store Connect; optionally creates a Team API key | Admin role |
| **Team uploader** (one student per team, with a Mac) | Uploads the team's builds with Transporter | **Developer** role on the UF team, limited to their app |
| **Students** | Write code and build | Access to the Expo project only |

## Part 1 · App Manager: set up a new app (once per app)

About 30 minutes the first time; a second app in the same Expo account reuses the certificate and key.

- [ ] **1. Get into the Expo project.** The project owner invites you on expo.dev (account settings → Members). Better: keep all UF apps in one **Expo organization** so credentials are shared automatically.
- [ ] **2. Check the project.**

- `app.json`: a unique `ios.bundleIdentifier` (e.g. `edu.ufl.gatorgrasp`), `"infoPlist": { "ITSAppUsesNonExemptEncryption": false }`, and a config plugin with permission text for every native feature (camera, NFC, location…).
- `.gitignore` must not hide `package.json`, `app.json` or `eas.json`. EAS drops ignored files even if committed; for a `*.json` rule add `!package.json`, `!app.json`, `!eas.json`.
- One lock file only (delete `package-lock.json` if the project uses bun).

- [ ] **3. Put env vars on EAS** so every build gets them:

```
npx eas-cli@latest env:push --environment production --path .env
```

- [ ] **4. First build, logged in to Apple.** Answer **yes** to the Apple login. EAS registers the bundle ID and makes the provisioning profile. When it asks about a distribution certificate, **reuse the existing one** (serial 793276B0…). If it offers only to generate one and fails at the 3-certificate limit, see "Certificates" in Part 1b.

```
npx eas-cli@latest build -p ios --profile production
```

- [ ] **5. Create the app record.** Run `npx eas-cli@latest submit -p ios --latest`. Once it prints "Prepared App Store Connect for…", the record exists; press **Ctrl+C** at the API key prompt. If the name was taken, EAS adds a suffix; rename under App Store Connect → **App Information**. Without a record, Transporter fails with "No suitable application records were found".
- [ ] **6. Testers.** Step 5 also creates a TestFlight group, **Team (Expo)**, filled with UF team members. Remove anyone not on the project, add the students, and **check the group's Settings → Build Distribution shows Automatic (EAS-made groups say "Automatic for Xcode Builds"; confirm it covers Transporter uploads with the first update, see Part 3)**. Internal testers must be users on the UF team; anyone else goes in **External Testing** (one-time Beta App Review, about a day).
- [ ] **7. Set up the team uploader.** Get the Apple ID email of one student on the team who has a Mac. A UF Admin adds them in App Store Connect → **Users and Access → +** with the **Developer** role and app access limited to this app (only Admins can add users). Send them Part 2's uploader steps.
- [ ] **8. Invite students to the Expo project** and send them Parts 2 and 3.
- [ ] **Optional: Team API key.** With a key from a UF Admin (App Store Connect → Users and Access → Integrations → App Store Connect API → Team Keys, access **App Manager**), EAS uploads by itself and teams can skip Transporter. Add it with `submit -p ios --`

`latest` → **no** to generating → **Team key**.

# Part 1b · App Manager: ongoing jobs

## Uploads are the team's job

Each team's uploader delivers their own builds with Transporter (Part 3). If an uploader leaves, have a UF Admin add a replacement (Part 1 step 7) and remove the old user.

## When to rerun a build logged in to Apple

Students always answer **no** to the Apple login. Run a build yourself with **yes** when something changes on Apple's side: a new bundle ID, a new capability (push notifications, NFC, Sign in with Apple…), or an expired certificate or profile.

## Certificates

The UF team can hold only **3** Apple Distribution certificates, shared by every UF app. **Never revoke one**: other projects depend on them. Certificates aren't tied to an app, so reuse yours.

- **Same Expo account or organization:** EAS offers the existing certificate automatically.
- **Different Expo account:** copy it over by hand.
  1. In a project that has it: `npx eas-cli@latest credentials -p ios` → **production** (not development) → **credentials.json → Download**. You get `credentials.json` (holds the password) and `credentials/ios/dist-cert.p12`.
  2. Copy the `.p12` into the new project's folder.
  3. In the new project: `credentials -p ios` → **production** → **Build Credentials** → **Distribution Certificate: Add a new one to your account** → "Generate new?" **no** → type `./dist-cert.p12` at the prompt → password → Team ID `8S5N9N3559` → use it for the project **yes** → generate a provisioning profile **yes**.
  4. Delete `dist-cert.p12`, `credentials.json` and the `credentials` folder everywhere. EAS keeps the certificate; re-download it anytime.

The current certificate expires **Oct 16, 2027**. Apps already in the App Store and TestFlight keep working; new builds then need a new certificate, which EAS can generate since expired ones stop counting toward the limit.

# Part 2 · Students: one-time setup

Works on Windows or Mac; builds run in the cloud, so you don't need Xcode.

- [ ] **1. Make an Expo account** at expo.dev and accept the App Manager's invite to the project.
- [ ] **2. Install the tools:** Node.js (LTS) and git, plus bun if the project has a `bun.lock`.
- [ ] **3. Get the code and install dependencies:**

```
git clone <repo url>
cd <app folder>
bun install
```

(Use `npm install` if the project has `package-lock.json` instead.)

- [ ] **4. Log in to Expo** and check you're the right account:

```
npx eas-cli@latest login
npx eas-cli@latest whoami
```

- [ ] **5. To test on your iPhone,** send the App Manager the email of your Apple ID so they can add you as a tester, then install the **TestFlight** app and accept the invite email.

You never need to sign in to an Apple developer account, and never answer **yes** to the Apple login prompt.

## Team uploader only (one student with a Mac)

1. Send the App Manager the email of your Apple ID. A UF Admin adds you to App Store Connect as a **Developer**, limited to your app.
2. Accept the App Store Connect invite email.
3. Install **Transporter** from the Mac App Store and sign in with that Apple ID.

The Developer role lets you upload builds; it doesn't change the rule above. Still answer **no** to the Apple login when building.

On a shared team Mac, each uploader signs in to Transporter with **their own student Apple ID**, not the App Manager's account, and signs out afterwards.

# Part 3 · Students: ship an update to TestFlight

1. **Pull the latest code** and install dependencies (`git pull`, then `bun install`).

2. **Build** (any student, any OS):

```
npx eas-cli@latest build -p ios --profile production --non-interactive
```

`--non-interactive` skips the Apple login and uses the stored certificate. Without it, answer **no** when asked.

3. **Wait** 10–20 minutes, plus any queue. Copy the build link the terminal prints (expo.dev/…/builds/…) and send it to your team's uploader.
4. **Uploader:** open the link, click **Download** to get the `.ipa`, then in **Transporter** drag it in and click **Deliver**.
5. **Test.** Apple processes the build in 5–30 minutes, then it appears in the TestFlight app for testers.

## Did the update reach testers?

Two things must happen for testers' phones to update.

1. **The build joins the tester group.** After Apple's "finished processing" email, open App Store Connect → your app → **TestFlight** → your internal group (e.g. **Team (Expo)**) → **Builds**. The new build number should be listed and the header count should go up (e.g. "1 Build" → "2 Builds").
   - Listed: distribution is automatic; nothing to do, now or for later uploads.
   - Not listed: click **+** and add it. The group's **Settings → Build Distribution** may then only cover Xcode builds, so this is needed after every upload; tell the App Manager.
2. **Phones install it.** Each tester opens the app's page in the **TestFlight** app and turns on **Automatic Updates**. Otherwise they get a notification and tap **Update**.

**Shared or lab Mac:** sign in to Transporter with the uploader's own student Apple ID (the one added as Developer), never the App Manager's. Whoever is signed in, anyone using that Mac can upload, and do anything else that account can, as them. Sign out of Transporter (Transporter → Sign Out) when you're done on a computer others use.

If the App Manager later adds a Team API key, add `--auto-submit` to the build command and skip steps 3–4.

Things to know:

- The build number goes up by itself. To change the version users see (1.0.0 → 1.1.0), edit `version` in `app.json`.

- A **new env var** must be on EAS before you build, or it will be empty in the app: ask the App Manager, or run `npx eas-cli@latest env:create --environment production`.
- A new library that uses the **camera, NFC, location or another permission** needs its config plugin and permission text in `app.json`, or the app crashes when it's used.
- A change to **Apple capabilities** (push notifications, Sign in with Apple…) or the **bundle ID** needs the App Manager to run the next build.
- If a build finished but didn't upload, run `npx eas-cli@latest submit -p ios --latest` (API key required).

## Troubleshooting

| Symptom | Cause | Fix | Who |
|---|---|---|---|
| "Failed to register bundle identifier" / Apple 403 | Answered **yes** to the Apple login | Rebuild with `--non-interactive`, or answer **no** | Student |
| Transporter: "No suitable application records were found" | App record missing, or the uploader's user lacks access to this app | Part 1 step 5; or have a UF Admin give the uploader access to the app | App Manager |
| Transporter sign-in works but the upload is refused | Uploader isn't on the UF team, or hasn't accepted the invite | Accept the App Store Connect invite; check the role is Developer or higher | Uploader |
| Transporter: build number already used | That build was already uploaded | Build again; the number goes up automatically | Student |
| "Generate a new App Store Connect API Key?" fails | Only Admins can create keys | Ctrl+C; upload with Transporter | Anyone |
| Build asks for certificates or fails on credentials | App not set up, or certificate expired | Rerun Part 1 step 4 | App Manager |
| "Maximum number of Distribution Certificates" | Team already has 3 | Don't revoke; reuse (Part 1b, Certificates) | App Manager |

| Symptom | Cause | Fix | Who |
|---|---|---|---|
| "There are no credentials configured for the AD_HOC distribution" | Picked the development profile | Pick **production** | Anyone |
| "package.json does not exist in /Users/expo/workingdir/..." | A `.gitignore` rule like `*.json` | Add `!package.json`, `!app.json`, `!eas.json`, commit, rebuild | Anyone |
| `git add --intent-to-add ... exited with non-zero code` | File is gitignored, or a stale `.git/index.lock` | Run the `git add` yourself to see why | Anyone |
| "No environment variables ... found" | Env vars not on EAS | `env:push` or `env:create`, rebuild | Anyone |
| App crashes right after opening | Missing env var or permission plugin | Fix `app.json` / env vars, rebuild | Student |
| Wrong Expo account / project not found | Logged in as someone else, or not invited | `logout`, `login`, `whoami`; `unset EXPO_TOKEN` if set | Student |
| "does not satisfy the CLI version constraint" | Old cached eas-cli | Use `npx eas-cli@latest` | Anyone |
| Build stuck on "new" | Free-plan queue | Wait; don't start duplicate builds | Anyone |